

PART 1: JAVASCRIPT INTERVIEW QUESTIONS

1. JavaScript Basics & Core Concepts

- What are JavaScript data types?
 - Difference between `var`, `let`, and `const`
 - Memory management in JavaScript
 - Hoisting in JavaScript
 - Temporal Dead Zone (TDZ)
 - `null` vs `undefined`
 - `==` vs `===`
 - Confirm vs Prompt
 - What is JavaScript?
 - How JavaScript works internally?
-

2. Functions & Execution Context

- Function declaration vs function expression
 - Arrow functions
 - IIFE (Immediately Invoked Function Expression)
 - Callback functions
 - First-class functions
 - Higher-order functions
 - Closures (theory + output-based questions)
 - `this` keyword (different scenarios)
-

3. Arrays, Objects & Iteration

- Objects and object methods
 - Array methods: `map`, `filter`, `reduce`, `forEach`
 - Array-specific loops (`for`, `for...of`, `for...in`)
 - Compare two arrays (code)
 - String-based questions
 - Array-based DSA questions
 - Star pattern problems
-

4. Asynchronous JavaScript

- What is asynchronous JavaScript?
- Promises (theory + examples)
- Async / Await

- Event loop
 - Callback vs Promise vs Async/Await
 - Fetch API
 - Output-based async questions
-

5. DOM & Events

- DOM manipulation
 - Event handling
 - Event bubbling & capturing
 - Practical DOM-based questions
-

6. Advanced & Interview-Focused Topics

- Polyfills
 - Output-based questions on:
 - Closures
 - Hoisting
 - Functions
 - Practical JavaScript tasks (logic-based)
-



PART 2: REACT INTERVIEW QUESTIONS

1. React Fundamentals

- What is React?
 - Why is React used?
 - Features of React
 - What is SPA (Single Page Application)?
 - Declarative programming in React
 - JSX
 - How JSX is converted into JavaScript
 - Virtual DOM vs Actual DOM
 - What is React Fiber?
 - Reconciliation
 - Diffing Algorithm
-

2. Components & Architecture

- What are components?
 - Functional vs Class components
 - Stateless vs Stateful components
 - Parent to child communication
 - Child to parent communication
 - Lifting state up
-

3. Props & State

- What are props?
 - Why are props read-only?
 - Props vs State
 - What is state?
 - Why React re-renders on state change?
 - Why should state not be updated directly?
 - Why state updates are asynchronous?
 - Why React batches state updates?
 - Why React skips re-render on same state value?
-

4. Hooks (Core Focus Area)

◇ **useState**

- Syntax of `useState`
- Functional state updater
- Updating array/object using spread operator
- Output-based questions on `useState`

◇ **useEffect**

- What is `useEffect`?
- Why dependency array is needed
- No dependency array behavior
- Empty dependency array behavior
- `useEffect` with props/state
- Infinite re-render loop scenario
- Cleanup function
- Why `useEffect` runs after render

◇ **useRef**

- What is `useRef`?
- `useRef` vs `useState`
- How `useRef` avoids re-renders

- DOM access using useRef
 - Storing mutable values
 - Child to parent communication using useRef
 - Character counter using textarea & useRef
-

5. Performance Optimization

- What is memoization?
 - Why memoization is used?
 - What is React.memo?
 - useCallback
 - useMemo
 - Difference between useCallback and useMemo
 - Preventing unnecessary re-renders
 - Parent re-render → child behavior
 - StrictMode
-

6. Lists & Conditional Rendering

- Rendering lists using `map`
 - Why `map()` is used in React
 - Why keys are required
 - Why index should not be used as key
 - Conditional rendering using:
 - `&&`
 - Ternary operator
 - Toggle show/hide component
-

7. Forms & Inputs

- Controlled component
 - Uncontrolled component
 - useRef vs controlled form
 - Handling input fields
 - Radio button → textbox scenario
 - Search filter input
-

8. Context API

- What is Context API?

- Provider & Consumer
 - When to use Context
 - Props drilling vs Context
 - useContext hook
 - Sharing user info using Context
-

9. Routing & API Integration

- What is React Router?
 - BrowserRouter, Routes, Route
 - Link vs <a> tag
 - Navigation without page reload
 - Fetch API using useEffect
 - Async/Await API call
 - Loading & error handling
 - Mapping API response to UI
-

10 State Management & Advanced Hooks

- useReducer
 - Counter using useReducer
 - Custom hooks
 - Redux (intro-level)
-

Machine Coding / Practical Tasks

- Counter using useState
- Todo App (add/delete)
- Show/hide toggle
- Character counter
- Fetch users & render list
- Dynamic checkbox counter
- Form handling (name/email)